

How Sessions & Authentication Work in Laravel

Before we write any code, I want to talk a little bit about how sessions work in Laravel. I don't want you to just learn the syntax and not understand what is actually happening under the hood. We will also use the `session` helper function to manually create session data.

Sessions

A session is a way to store information across multiple requests in your application. HTTP on it's own is a stateless protocol. This means that each request is independent of the previous request. This is great for performance, but it can be a problem when you need to maintain state information across multiple requests.

In Laravel and in many other frameworks and web apps, sessions are used to maintain state information, such as user authentication status, flash messages, etc. We've already seen an example of flash messages.

Authentication & Sessions

When a user logs in, Laravel will create a session for that user and store the user's authentication status in the session. When the user logs out, Laravel will destroy the session.

If we submit a new job listing, ultimately we want to get the user ID from the session and store it in the database with the listing. Then when we have functionality like updating and deleting listings, we want to check if the user is logged in and if they are the owner of the listing.

Session Cookies

As I said, when a user logs in, Laravel will create a session for that user and store the user's authentication status in the session. This session is stored in a cookie on the user's browser. The cookie contains the session ID, which is a unique identifier for the session. The session ID is sent to the server with each request. The server uses the session ID to retrieve the session data from the database.

By default, Laravel uses a cookie named `laravel_session` or `your_project_name_session` to store the session ID. You can open your browser's developer tools and see the cookie. The data will be encrypted and signed to prevent tampering. Your app has a key defined in the `.env` file that is used to encrypt and sign the data. It has the key `APP_KEY` and the value is a random string of characters.

Remember Me Cookie

If the "Remember Me" option is enabled during login, Laravel will generate a long-lived cookie called `remember_token`. This token is stored in the `remember_token` field of the users table. If the session expires (e.g., the browser is closed), Laravel can automatically log the user back in using this cookie the next time they visit the site.

CSRF Tokens

Laravel also uses a CSRF token to prevent cross-site request forgery attacks. This token is stored in the session and is sent to the server with each request. The server uses the token to verify that the request is legitimate.

Session Configuration & Database Setup

Sessions are configured in the `config/session.php` file. Laravel supports different session drivers such as file, cookie, database, memcached, redis, and array. We are using the `database` driver in this project.

If you look in your database using PG Admin or something else, you will see a table called `sessions`. This is where Laravel stores the session data. There is a field called `payload` that contains the session data. This data is serialized and encrypted before being stored in the database. This data could be anything from user authentication status to CSRF tokens to flash messages.

There is also a `user_id` field that is used to associate the session with a user. This is used to determine if the user is logged in or not. Right now, any sessions you have will not have a `user_id` because we have not logged in yet.

session Helper Function

We can manually create session data. To do this, we can use the `session` helper function.

Let's do this in our `HomeController`. It doesn't matter where you put this code because we aren't keeping it, but I will put it in the `index()` method.

```
session()->put('test', '123');  
$value = session()->get('test');  
dd($value);
```

Now when you go to the homepage, you should see `123`. This is how you can manually create session data.

If you look in the database, you this data will be in the session table but it will be encrypted. Also, a single record can have multiple values. You can copy the payload and paste it into a site like <https://www.base64decode.org/> or even ChatGPT to see what the data looks like.

You can now delete the session data by calling the `forget()` method.

```
session()->put('hello', 'world');  
session()->put('test', '123');  
session()->forget('test');  
$value = session()->get('test'); // This will return 'world'  
dd($value);
```

Now you will see `null` because the `test` key has been forgotten.

Delete that code.

So that is how sessions work in Laravel.